

Programacao Web

Ficha Pratica

Backend com Laravel + SQLite

API REST de Gestao de Restaurante

Framework	Laravel 11
Base de Dados	SQLite (local, sem servidor)
Autenticacao	Laravel Sanctum (tokens)
Papeis	Visitante Cliente Proprietario
Modelos	User, Category, Dish, Reservation
Ferramenta de Testes	Postman

Introducao

Nesta ficha vais construir uma API REST completa para um sistema de restaurante. A API permite consultar o menu, gerir pratos por categorias e tratar reservas de mesa — com tres niveis de acesso que reflectem um caso de uso real.

INFO Tres niveis de acesso

Visitante (nao autenticado) pode consultar categorias e o menu.

Cliente (autenticado) pode gerir o seu perfil e as suas reservas (criar, ver, editar, cancelar).

Proprietario (admin) tem acesso total: gere o menu, confirma/rejeita reservas e utilizadores.

O que vais construir

- Autenticacao com tokens Sanctum e edicao de perfil (nome, email, password, telefone)
- Categorias de pratos (Entradas, Pratos Principais, Sobremesas...)
- Menu com pratos associados a categorias e filtros por preco/disponibilidade
- Reservas com ciclo de vida: pending -> confirmed -> cancelled
- Autorizacao granular por papel em cada rota
- Relacao muitos-para-muitos entre reservas e pratos (pivot com quantidade)

INFO Comparacao com Node.js

Ja sabes: criar APIs REST, autenticacao JWT, relacoes em Mongoose.

Vais ver: como o Laravel organiza tudo com migrations, Eloquent ORM, recursos e middleware.

A logica e a mesma. As convencoes e a sintaxe e que mudam.

Estrutura de Fases

1. Configuracao do projecto
2. Base de dados: migracoes, modelos e seeders
3. Autenticacao, registo e gestao de perfil
4. CRUD do menu (categorias e pratos)
5. Reservas: ciclo de vida e autorizacao

Fase 1 - Configuracao do Projecto

1.1 Criar o projecto

```
composer create-project laravel/laravel restaurante-api
cd restaurante-api
```

DICA Sem composer global?

Usa: php composer.phar create-project laravel/laravel restaurante-api

1.2 Configurar o SQLite

Abre o ficheiro .env na raiz do projecto e substitui toda a configuracao DB_* por:

```
DB_CONNECTION=sqlite
# Remove ou comenta: DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, DB_PASSWORD
```

Cria o ficheiro da base de dados:

```
# Windows PowerShell:
New-Item -ItemType File -Path "database/database.sqlite"

# Git Bash / WSL / Mac:
touch database/database.sqlite
```

INFO Como funciona o SQLite no Laravel

O Laravel encontra automaticamente o ficheiro em database/database.sqlite.

Nao precisas de instalar nenhum servidor. O PHP ja inclui suporte nativo a SQLite.

1.3 Instalar o Sanctum e verificar

```
composer require laravel/sanctum
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"
php artisan migrate

# Confirmar versao do Laravel:
php artisan --version
# Deve mostrar: Laravel Framework 11.x.x
```

1.4 Iniciar o servidor

```
php artisan serve
# API disponivel em: http://127.0.0.1:8000/api
```

AVISO Terminal dedicado ao servidor

Mantem este terminal aberto enquanto testes no Postman.

Abre um segundo terminal para correr os comandos artisan seguintes.

Fase 2 - Base de Dados: Migracoes, Modelos e Seeders

2.1 Diagrama de relacoes

NOTA Relacoes entre entidades

users (1) ---- (N) reservations Um utilizador tem varias reservas.
categories (1) ---- (N) dishes Uma categoria tem varios pratos.
reservations (N) -- (N) dishes Uma reserva pode ter varios pratos.
Tabela pivot: reservation_dish (com quantity).

2.2 Criar as migracoes

```
php artisan make:migration create_categories_table
php artisan make:migration create_dishes_table
php artisan make:migration create_reservations_table
php artisan make:migration create_reservation_dish_table
php artisan make:migration add_role_to_users_table --table=users
```

Migracao: categories

```
// database/migrations/xxxx_create_categories_table.php
public function up(): void
{
    Schema::create('categories', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->text('description')->nullable(); // Ex: 'Entradas', 'Sobremesas'
        $table->boolean('active')->default(true);
        $table->timestamps();
    });
}
```

Migracao: dishes

```
// database/migrations/xxxx_create_dishes_table.php
public function up(): void
{
    Schema::create('dishes', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->text('description')->nullable();
        $table->decimal('price', 8, 2); // Preco com 2 casas decimais
        $table->boolean('available')->default(true);
        $table->string('allergens')->nullable(); // Ex: 'gluten, lactose'
        $table->foreignId('category_id')
            ->constrained() // Referencia 'categories'
            ->onDelete('cascade');
        $table->timestamps();
    });
}
```

Migracao: reservations

```
// database/migrations/xxxx_create_reservations_table.php
public function up(): void
{
    Schema::create('reservations', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->onDelete('cascade');
        $table->dateTime('reserved_at'); // Data e hora da reserva
        $table->integer('guests'); // Numero de pessoas
        $table->text('notes')->nullable(); // Pedidos especiais
        $table->enum('status', ['pending', 'confirmed', 'cancelled'])
    });
}
```

```

        ->default('pending');
    \ $table->timestamps();
});
}

```

Migracao: reservation_dish (pivot N:N)

```

// database/migrations/xxxx_create_reservation_dish_table.php
public function up(): void
{
    Schema::create('reservation_dish', function (Blueprint $table) {
        $table->id();
        $table->foreignId('reservation_id')->constrained()->onDelete('cascade');
        $table->foreignId('dish_id')->constrained()->onDelete('cascade');
        $table->integer('quantity')->default(1);
    });
}

```

Migracao: adicionar role e phone aos utilizadores

```

// database/migrations/xxxx_add_role_to_users_table.php
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->string('role')->default('client'); // 'client' ou 'admin'
        $table->string('phone')->nullable();
    });
}
public function down(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->dropColumn(['role', 'phone']);
    });
}

```

Corre todas as migracoes:

```
php artisan migrate
```

2.3 Criar os Modelos Eloquent

```

php artisan make:model Category
php artisan make:model Dish
php artisan make:model Reservation

```

Modelo: Category (app/Models/Category.php)

```

<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Category extends Model
{
    protected $fillable = ['name', 'description', 'active'];
    protected $casts = ['active' => 'boolean'];

    public function dishes(): HasMany
    {
        return $this->hasMany(Dish::class);
    }
}

```

Modelo: Dish (app/Models/Dish.php)

```
<?php
```

```

namespace App\Models;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\{BelongsTo, BelongsToMany};

class Dish extends Model
{
    protected $fillable =
['name', 'description', 'price', 'available', 'allergens', 'category_id'];
    protected $casts = ['available'=>'boolean', 'price'=>'decimal:2'];

    public function category(): BelongsTo
    {
        return $this->belongsTo(Category::class);
    }

    // Um prato pode aparecer em varias reservas
    public function reservations(): BelongsToMany
    {
        return $this->belongsToMany(Reservation::class)->withPivot('quantity');
    }
}

```

Modelo: Reservation (app/Models/Reservation.php)

```

<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\{BelongsTo, BelongsToMany};

class Reservation extends Model
{
    protected $fillable = ['user_id', 'reserved_at', 'guests', 'notes', 'status'];
    protected $casts = ['reserved_at' => 'datetime'];

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    // Uma reserva pode incluir varios pratos (com quantidade)
    public function dishes(): BelongsToMany
    {
        return $this->belongsToMany(Dish::class)->withPivot('quantity');
    }
}

```

Modelo: User (app/Models/User.php) — atualizar

Abre o ficheiro existente e atualiza para incluir o Sanctum e os novos campos:

```

<?php
namespace App\Models;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens;
use Illuminate\Database\Eloquent\Relations\HasMany;

class User extends Authenticatable
{
    use HasApiTokens, Notifiable;

    protected $fillable = ['name', 'email', 'password', 'role', 'phone'];
    protected $hidden = ['password', 'remember_token'];
    protected $casts = ['email_verified_at'=>'datetime', 'password'=>'hashed'];

    // Metodo de conveniencia para verificar papel
    public function isAdmin(): bool
    {
        return $this->role === 'admin';
    }
}

```

```

public function reservations(): HasMany
{
    return $this->hasMany(Reservation::class);
}

```

2.4 Seeder com dados de teste

Edita database/seeder/DatabaseSeeder.php:

```

<?php
namespace Database\Seeders;
use App\Models\{Category,Dish,Reservation,User};
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;

class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        // Utilizadores
        \User::create([
            'name' => 'Proprietario',
            'email' => 'admin@restaurante.pt',
            'password' => Hash::make('password123'),
            'role' => 'admin',
            'phone' => '912345678',
        ]);
        \User::create([
            'name' => 'Maria Oliveira',
            'email' => 'maria@email.com',
            'password' => Hash::make('password123'),
            'role' => 'client',
            'phone' => '961234567',
        ]);

        // Categorias e pratos
        \Category::create(['name'=>'Entradas','description'=>'Para comecar bem']);
        \Category::create(['name'=>'Pratos Principais','description'=>'O melhor do chef']);
        \Category::create(['name'=>'Sobremesas','description'=>'Doce final']);

        \Dish::create(['name'=>'Caldo Verde','price'=>5.50,'allergens'=>'lactose']);
        \Dish::create(['name'=>'Rissois de Camarao','price'=>7.00,'allergens'=>'gluten,marisco']);
        \Dish::create(['name'=>'Bacalhau a Bras','price'=>18.50,'allergens'=>'ovos,gluten']);
        \Dish::create(['name'=>'Bife na Pedra','price'=>22.00,'allergens'=>'']);
        \Dish::create(['name'=>'Mousse de Chocolate','price'=>4.50,'allergens'=>'ovos,lactose']);

        // Reserva de exemplo
        \Reservation::create([
            'user_id' => \User::find(2)->id,
            'reserved_at' => now()->addDays(3),
            'guests' => 2,
            'notes' => 'Mesa perto da janela, se possivel.',
            'status' => 'pending',
        ]);
        \Reservation::attach([
            \Dish::find(1)->id => ['quantity' => 2],
            \Dish::find(2)->id => ['quantity' => 2],
        ]);
    }
}

```

```
php artisan db:seed
```

```
# Para recomencar do zero a qualquer momento:
```

```
php artisan migrate:fresh --seed
```

DICA Verificar dados com Tinker

```
php artisan tinker
```

```
>>> App\Models\Dish::with('category')->get();
```

```
>>> App\Models\Reservation::with(['user','dishes'])->get();
```

```
>>> App\Models\User::all(['id','name','email','role']);
```


Fase 3 - Autenticacao e Gestao de Perfil

3.1 Criar o AuthController

```
php artisan make:controller AuthController
```

```
<?php
namespace App\Http\Controllers;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\ValidationException;

class AuthController extends Controller
{
    // POST /api/register
    public function register(Request $request)
    {
        $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users',
            'password' => 'required|string|min:8|confirmed',
            'phone' => 'nullable|string|max:20',
        ]);
        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
            'phone' => $request->phone,
            'role' => 'client', // Registro sempre cria cliente
        ]);
        $token = $user->createToken('auth_token')->plainTextToken;
        return response()->json(['message'=>'Conta criada.', 'token'=>$token, 'user'=>$user], 201);
    }

    // POST /api/login
    public function login(Request $request)
    {
        $request->validate(['email'=>'required|email', 'password'=>'required']);
        $user = User::where('email', $request->email)->first();
        if (!$user || !Hash::check($request->password, $user->password)) {
            throw ValidationException::withMessages(['email'=>['Credenciais invalidas.']] );
        }
        $user->tokens()->delete(); // Invalida sessoes anteriores
        $token = $user->createToken('auth_token')->plainTextToken;
        return response()->json(['message'=>'Login efectuado.', 'token'=>$token, 'user'=>$user]);
    }

    // POST /api/logout
    public function logout(Request $request)
    {
        $request->user()->currentAccessToken()->delete();
        return response()->json(['message'=>'Sessao terminada.']);
    }

    // GET /api/me
    public function me(Request $request)
    {
        return response()->json($request->user());
    }

    // PUT /api/me - o cliente edita o seu proprio perfil
    public function updateMe(Request $request)
    {
        $user = $request->user();
        $data = $request->validate([
            'name' => 'sometimes|string|max:255',
            'email' => 'sometimes|email|unique:users,email,.'.($user->id),
            'phone' => 'nullable|string|max:20',
            'password' => 'sometimes|string|min:8|confirmed',
        ]);
    }
}
```

```

    });
    if (isset(\$_data['password'])) {
        \$_data['password'] = Hash::make(\$_data['password']);
    }
    \$_user->update(\$_data);
    return response()->json(['message'=>'Perfil atualizado.', 'user'=>\$_user]);
}
}

```

3.2 Criar o middleware IsAdmin

```
php artisan make:middleware IsAdmin
```

```

// app/Http/Middleware/IsAdmin.php
public function handle(Request \$request, Closure \$next): Response
{
    if (!\$request->user()?->isAdmin()) {
        return response()->json(['message'=>'Acesso reservado ao proprietario.'], 403);
    }
    return \$next(\$request);
}

```

Regista o alias em bootstrap/app.php:

```

->withMiddleware(function (Middleware \$middleware) {
    \$middleware->alias([
        'is_admin' => \App\Http\Middleware\IsAdmin::class,
    ]);
})

```

3.3 Definir as rotas de autenticao (routes/api.php)

```

<?php
use App\Http\Controllers\AuthController;
use Illuminate\Support\Facades\Route;

// Rotas publicas
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login', [AuthController::class, 'login']);

// Rotas autenticadas
Route::middleware('auth:sanctum')->group(function () {

    // Perfil
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::get('/me', [AuthController::class, 'me']);
    Route::put('/me', [AuthController::class, 'updateMe']);

    // (Rotas de menu e reservas sao adicionadas nas fases seguintes)
});

```

3.4 Testar no Postman

DICA Configuracao do Postman

1. Cria uma Collection 'Restaurante API'.
2. Cria um Environment 'Local' com base_url = http://127.0.0.1:8000/api
3. Adiciona a variavel 'token' (vazia).
4. Apos o login, copia o token da resposta para a variavel token.
5. Nas rotas protegidas: Authorization > Bearer Token > {{token}}

Método	Endpoint	Acesso	Descrição
POST	/api/register	Público	Criar conta de cliente
POST	/api/login	Público	Login — devolve token
GET	/api/me	Cliente	Ver o meu perfil
PUT	/api/me	Cliente	Editar nome, email, password, telefone
POST	/api/logout	Cliente	Terminar sessão

Exemplo de body para PUT /api/me (apenas os campos a alterar):

```
{
  "name": "Maria Oliveira Santos",
  "phone": "968888999",
  "password": "novaPassword456",
  "password_confirmation": "novaPassword456"
}
```

Fase 4 - CRUD do Menu (Categorias e Pratos)

4.1 Criar os controllers

```
php artisan make:controller CategoryController --resource
php artisan make:controller DishController --resource
```

INFO O que é --resource?

Gera automaticamente os métodos index, store, show, update e destroy.

É o equivalente ao `router.route('/dishes').get().post()` do Express, mas estruturado.

4.2 CategoryController (app/Http/Controllers/CategoryController.php)

```
<?php
namespace App\Http\Controllers;
use App\Models\Category;
use Illuminate\Http\Request;

class CategoryController extends Controller
{
    // GET /api/categories - PUBLICO
    public function index()
    {
        return response()->json(
            Category::withCount('dishes')->where('active', true)->get()
        );
    }

    // GET /api/categories/{id} - PUBLICO (inclui pratos disponiveis)
    public function show(Category $category)
    {
        return response()->json(
            $category->load(['dishes' => fn($q) => $q->where('available', true)])
        );
    }

    // POST /api/categories - ADMIN
    public function store(Request $request)
    {
        $data = $request->validate([
            'name' => 'required|string|max:100|unique:categories',
            'description' => 'nullable|string',
            'active' => 'boolean',
        ]);
        return response()->json(Category::create($data), 201);
    }

    // PUT /api/categories/{id} - ADMIN
    public function update(Request $request, Category $category)
    {
        $data = $request->validate([
            'name' => 'sometimes|string|max:100|unique:categories,name,'.($category->id),
            'description' => 'nullable|string',
            'active' => 'boolean',
        ]);
        $category->update($data);
        return response()->json($category);
    }

    // DELETE /api/categories/{id} - ADMIN
    public function destroy(Category $category)
    {
        $category->delete();
        return response()->json(['message'=>'Categoria removida.']);
    }
}
```

4.3 DishController (app/Http/Controllers/DishController.php)

```
<?php
namespace App\Http\Controllers;
use App\Models\Dish;
use Illuminate\Http\Request;

class DishController extends Controller
{
    // GET /api/dishes?category_id=1&available=true&max_price=15 - PUBLICO
    public function index(Request $request)
    {
        $q = Dish::with('category');
        if ($request->has('category_id')) {
            $q->where('category_id', $request->category_id);
        }
        if ($request->has('available')) {
            $q->where('available', $request->boolean('available'));
        }
        if ($request->has('max_price')) {
            $q->where('price', '<=', $request->max_price);
        }
        return response()->json($q->paginate(12));
    }

    // GET /api/dishes/{id} - PUBLICO
    public function show(Dish $dish)
    {
        return response()->json($dish->load('category'));
    }

    // POST /api/dishes - ADMIN
    public function store(Request $request)
    {
        $data = $request->validate([
            'name' => 'required|string|max:255',
            'description' => 'nullable|string',
            'price' => 'required|numeric|min:0',
            'available' => 'boolean',
            'allergens' => 'nullable|string',
            'category_id' => 'required|exists:categories,id',
        ]);
        $dish = Dish::create($data);
        return response()->json($dish->load('category'), 201);
    }

    // PUT /api/dishes/{id} - ADMIN
    public function update(Request $request, Dish $dish)
    {
        $data = $request->validate([
            'name' => 'sometimes|string|max:255',
            'description' => 'nullable|string',
            'price' => 'sometimes|numeric|min:0',
            'available' => 'boolean',
            'allergens' => 'nullable|string',
            'category_id' => 'sometimes|exists:categories,id',
        ]);
        $dish->update($data);
        return response()->json($dish->load('category'));
    }

    // DELETE /api/dishes/{id} - ADMIN
    public function destroy(Dish $dish)
    {
        $dish->delete();
        return response()->json(['message'=>'Prato removido.']);
    }
}
```

4.4 Adicionar rotas do menu (routes/api.php)

```

use App\Http\Controllers\{CategoryController, DishController};

// --- PUBLICAS (fora do grupo auth:sanctum) -----
Route::get('categories', [CategoryController::class, 'index']);
Route::get('categories/{category}', [CategoryController::class, 'show']);
Route::get('dishes', [DishController::class, 'index']);
Route::get('dishes/{dish}', [DishController::class, 'show']);

// --- Dentro do grupo auth:sanctum -----
Route::middleware('auth:sanctum')->group(function () {

    // Perfil (ja existente)
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::get('/me', [AuthController::class, 'me']);
    Route::put('/me', [AuthController::class, 'updateMe']);

    // Menu - apenas proprietario pode gerir
    Route::middleware('is_admin')->group(function () {
        Route::post('categories', [CategoryController::class, 'store']);
        Route::put('categories/{category}', [CategoryController::class, 'update']);
        Route::delete('categories/{category}', [CategoryController::class, 'destroy']);

        Route::post('dishes', [DishController::class, 'store']);
        Route::put('dishes/{dish}', [DishController::class, 'update']);
        Route::delete('dishes/{dish}', [DishController::class, 'destroy']);
    });
});

```

4.5 Referencia de endpoints — Menu

Método	Endpoint	Acesso	Descrição
GET	/api/categories	Público	Lista categorias activas com contagem de pratos
GET	/api/categories/{id}	Público	Categoria + pratos disponiveis
POST	/api/categories	Proprietario	Criar categoria
PUT	/api/categories/{id}	Proprietario	Editar categoria
DELETE	/api/categories/{id}	Proprietario	Apagar (apaga pratos em cascata)
GET	/api/dishes	Público	Listar pratos (filtros: category_id, available, max_price)
GET	/api/dishes/{id}	Público	Detalhes de um prato + categoria
POST	/api/dishes	Proprietario	Criar prato
PUT	/api/dishes/{id}	Proprietario	Editar prato
DELETE	/api/dishes/{id}	Proprietario	Apagar prato

Fase 5 - Reservas: Ciclo de Vida e Autorizacao

5.1 Ciclo de vida de uma reserva

NOTA Estados e transicoes possiveis

[pending] ---- proprietario confirma ----> [confirmed]

[pending] ---- proprietario rejeita ----> [cancelled]

[pending] ---- cliente cancela ----> [cancelled]

[confirmed] ---- proprietario rejeita ----> [cancelled]

Regra importante: cliente NAO pode editar uma reserva ja confirmada.

Regra importante: cada cliente so ve e edita as suas proprias reservas.

5.2 ReservationController

```
php artisan make:controller ReservationController
```

```
<?php
namespace App\Http\Controllers;
use App\Models\Reservation;
use Illuminate\Http\Request;

class ReservationController extends Controller
{
    // GET /api/reservations
    // Admin ve todas. Cliente ve apenas as suas.
    public function index(Request $request)
    {
        $user = $request->user();
        $query = Reservation::with(['user', 'dishes']);

        if (!$user->isAdmin()) {
            $query->where('user_id', $user->id);
        }
        if ($request->has('status')) {
            $query->where('status', $request->status);
        }
        return response()->json($query->orderBy('reserved_at')->paginate(10));
    }

    // GET /api/reservations/{id}
    public function show(Request $request, Reservation $reservation)
    {
        $this->checkAccess($request->user(), $reservation);
        return response()->json($reservation->load(['user', 'dishes']));
    }

    // POST /api/reservations - Cliente cria reserva
    public function store(Request $request)
    {
        $data = $request->validate([
            'reserved_at' => 'required|date|after:now',
            'guests' => 'required|integer|min:1|max:20',
            'notes' => 'nullable|string|max:500',
            'dishes' => 'nullable|array',
            'dishes.*.id' => 'exists:dishes,id',
            'dishes.*.quantity' => 'integer|min:1',
        ]);

        $reservation = Reservation::create([
            'user_id' => $request->user()->id,
            'reserved_at' => $data['reserved_at'],
            'guests' => $data['guests'],
        ]);
    }
}
```

```

        'notes'      => \${data['notes']} ?? null,
        'status'     => 'pending',
    });

    // Associa pratos com quantidade (tabela pivot)
    if (!empty(\${data['dishes']})) {
        \${pivot} = collect(\${data['dishes']})
            ->mapWithKeys(fn(\${d}) => [\${d['id']} => ['quantity' => \${d['quantity']} ??
1]]);
        \${reservation->dishes()->sync(\${pivot});
    }

    return response()->json(\${reservation->load(['user','dishes']), 201);
}

// PUT /api/reservations/{id} - Cliente edita (so se pending)
public function update(Request \${request}, Reservation \${reservation})
{
    \${this->checkAccess}(\${request->user()}, \${reservation});

    if (\${reservation->status} !== 'pending') {
        return response()->json([
            'message' => 'So e possivel editar reservas pendentes.',
            ], 422);
    }

    \${data} = \${request->validate([
        'reserved_at'      => 'sometimes|date|after:now',
        'guests'           => 'sometimes|integer|min:1|max:20',
        'notes'            => 'nullable|string|max:500',
        'dishes'           => 'nullable|array',
        'dishes.*.id'      => 'exists:dishes,id',
        'dishes.*.quantity' => 'integer|min:1',
    ]]);

    \${reservation->update}(collect(\${data})->except('dishes')->toArray());

    if (isset(\${data['dishes']})) {
        \${pivot} = collect(\${data['dishes']})
            ->mapWithKeys(fn(\${d}) => [\${d['id']} => ['quantity' => \${d['quantity']} ??
1]]);
        \${reservation->dishes()->sync(\${pivot});
    }

    return response()->json(\${reservation->load(['user','dishes']));
}

// PATCH /api/reservations/{id}/cancel - Cliente cancela a sua reserva
public function cancel(Request \${request}, Reservation \${reservation})
{
    \${this->checkAccess}(\${request->user()}, \${reservation});

    if (\${reservation->status} === 'cancelled') {
        return response()->json(['message' => 'Reserva ja cancelada.'], 422);
    }
    \${reservation->update}(['status' => 'cancelled']);
    return response()->json(['message'=>'Reserva
cancelada.','reservation'=>\${reservation}]);
}

// PATCH /api/reservations/{id}/status - Admin altera estado
public function updateStatus(Request \${request}, Reservation \${reservation})
{
    \${request->validate([
        'status' => 'required|in:confirmed,cancelled',
    ]]);
    \${reservation->update}(['status' => \${request->status}]);
    return response()->json([
        'message'      => 'Estado atualizado.',
        'reservation' => \${reservation->load(['user','dishes']),
    ]]);
}

```



```
// DELETE /api/reservations/{id} - Admin apaga
public function destroy(Reservation $reservation)
{
    $reservation->delete();
    return response()->json(['message' => 'Reserva eliminada.']);
}

// Metodo privado: verifica se o utilizador pode aceder a esta reserva
private function checkAccess($user, Reservation $reservation): void
{
    if (!$user->isAdmin() && $reservation->user_id !== $user->id) {
        abort(403, 'Nao tens permissao para aceder a esta reserva.');
```

5.3 Adicionar rotas de reservas

Completa o routes/api.php com as rotas de reservas dentro do grupo auth:sanctum:

```
use App\Http\Controllers\ReservationController;

Route::middleware('auth:sanctum')->group(function () {

    // ... (rotas anteriores de perfil e menu) ...

    // Reservas - qualquer cliente autenticado:
    Route::get('reservations',
[ReservationController::class, 'index']);
    Route::post('reservations',
[ReservationController::class, 'store']);
    Route::get('reservations/{reservation}',
[ReservationController::class, 'show']);
    Route::put('reservations/{reservation}',
[ReservationController::class, 'update']);
    Route::patch('reservations/{reservation}/cancel',
[ReservationController::class, 'cancel']);

    // Reservas - apenas proprietario:
    Route::middleware('is_admin')->group(function () {

Route::patch('reservations/{reservation}/status', [ReservationController::class, 'updateStatus
']);
    Route::delete('reservations/{reservation}',
[ReservationController::class, 'destroy']);
    });
});
```

5.4 Referencia de endpoints — Reservas

Método	Endpoint	Acesso	Descrição
GET	/api/reservations	Cliente	Cliente: as suas reservas. Admin: todas (filtro ?status=pending)
POST	/api/reservations	Cliente	Criar reserva (com pratos opcionais)
GET	/api/reservations/{id}	Cliente	Ver detalhes (cliente: so as suas)
PUT	/api/reservations/{id}	Cliente	Editar reserva (so se status=pending)
PATCH	/api/reservations/{id}/cancel	Cliente	Cancelar a propria reserva
PATCH	/api/reservations/{id}/status	Proprietario	Confirmar ou rejeitar reserva
DELETE	/api/reservations/{id}	Proprietario	Eliminar reserva permanentemente

5.5 Exemplos de teste no Postman

Criar reserva com pratos (POST /api/reservations)

```
{
  "reserved_at": "2025-09-20 20:00:00",
  "guests": 3,
  "notes": "Aniversario — mesa com decoracao, por favor.",
  "dishes": [
    { "id": 1, "quantity": 3 },
    { "id": 3, "quantity": 3 }
  ]
}
```

Confirmar reserva — Admin (PATCH /api/reservations/1/status)

```
{ "status": "confirmed" }
```

Cenarios de autorizacao a verificar

Acção	Resposta esperada	Porquê
GET /api/dishes sem token	200 OK	Rota publica — visitantes podem ver o menu
POST /api/dishes sem token	401 Unauthorized	Gestao do menu requer autenticacao
POST /api/dishes com token de cliente	403 Forbidden	Apenas o proprietario gere o menu
POST /api/dishes com token de admin	201 Created	Proprietario tem acesso total ao menu
GET /api/reservations com token cliente	200 OK	Cliente ve apenas as suas proprias reservas
GET reserva de outro cliente	403 Forbidden	checkAccess() bloqueia o acesso — correcto
PUT reserva ja confirmada	422 Unprocessable	Logica de negocio impede edicao pos-confirmacao
PATCH /status com token de cliente	403 Forbidden	So o proprietario pode alterar o estado

Referencia Rapida — Todos os Endpoints

Autenticacao e Perfil

Método	Endpoint	Acesso	Descrição
POST	/api/register	Público	Criar conta de cliente
POST	/api/login	Público	Login — devolve token Sanctum
GET	/api/me	Cliente	Ver o meu perfil
PUT	/api/me	Cliente	Actualizar dados pessoais e password
POST	/api/logout	Cliente	Invalidar token actual

Menu — Categorias e Pratos

Método	Endpoint	Acesso	Descrição
GET	/api/categories	Público	Listar categorias activas
GET	/api/categories/{id}	Público	Categoria + pratos disponiveis
POST	/api/categories	Proprietario	Criar categoria
PUT	/api/categories/{id}	Proprietario	Editar categoria
DELETE	/api/categories/{id}	Proprietario	Apagar categoria
GET	/api/dishes	Público	Listar pratos (filtros: category_id, available, max_price)
GET	/api/dishes/{id}	Público	Detalhes de um prato
POST	/api/dishes	Proprietario	Criar prato
PUT	/api/dishes/{id}	Proprietario	Editar prato
DELETE	/api/dishes/{id}	Proprietario	Apagar prato

Reservas

Método	Endpoint	Acesso	Descrição
GET	/api/reservations	Cliente	Listar (cliente: so as suas; admin: todas + filtro status)
POST	/api/reservations	Cliente	Criar reserva com pratos opcionais
GET	/api/reservations/{id}	Cliente	Ver detalhes de uma reserva
PUT	/api/reservations/{id}	Cliente	Editar reserva (so se pending)
PATCH	/api/reservations/{id}/cancel	Cliente	Cancelar a propria reserva
PATCH	/api/reservations/{id}/status	Proprietario	Confirmar ou rejeitar reserva
DELETE	/api/reservations/{id}	Proprietario	Eliminar reserva

Comandos Artisan mais usados

Comando	O que faz
<code>php artisan serve</code>	Inicia o servidor de desenvolvimento
<code>php artisan migrate</code>	Corre as migracoes pendentes
<code>php artisan migrate:fresh --seed</code>	Apaga tudo e recria com dados de teste
<code>php artisan make:model Foo -m</code>	Cria modelo + migracao em simultaneo
<code>php artisan make:controller FooController --resource</code>	Controller com todos os metodos REST

Comando	O que faz
<code>php artisan make:middleware NomeMiddleware</code>	Cria um middleware
<code>php artisan route:list</code>	Lista todas as rotas registadas
<code>php artisan tinker</code>	Consola interactiva PHP
<code>php artisan db:seed</code>	Corre seeders sem apagar dados
<code>php artisan config:clear</code>	Limpa a cache de configuracao

Erros comuns e solucoes

AVISO SQLSTATE: no such table

Causa: Ficheiro .sqlite nao existe ou migracoes nao correram.

Solucao: `touch database/database.sqlite && php artisan migrate`

AVISO 401 Unauthenticated em rota protegida

Causa: Token em falta ou invalido no Postman.

Solucao: Authorization > Bearer Token > cola o token devolvido pelo login.

AVISO 422 — reserved_at must be after now

Causa: Data da reserva esta no passado.

Solucao: Usa uma data futura. Exemplo: 2025-09-20 20:00:00

AVISO 403 ao aceder a reserva de outro utilizador

Causa: O metodo `checkAccess()` bloqueou o acesso — esta correcto!

Solucao: Faz login com o utilizador dono da reserva e usa o seu token.

AVISO 422 ao editar reserva confirmada

Causa: Logica de negocio impede edicao apos confirmacao — correcto.

Solucao: So e possivel editar reservas com `status=pending`.